

Default Credentials Scanning

LLM-supported application-level default
credential scanner

Laurenz Pepe Simon & Mathis Zscheischler

**Design IT.
Create Knowledge.**

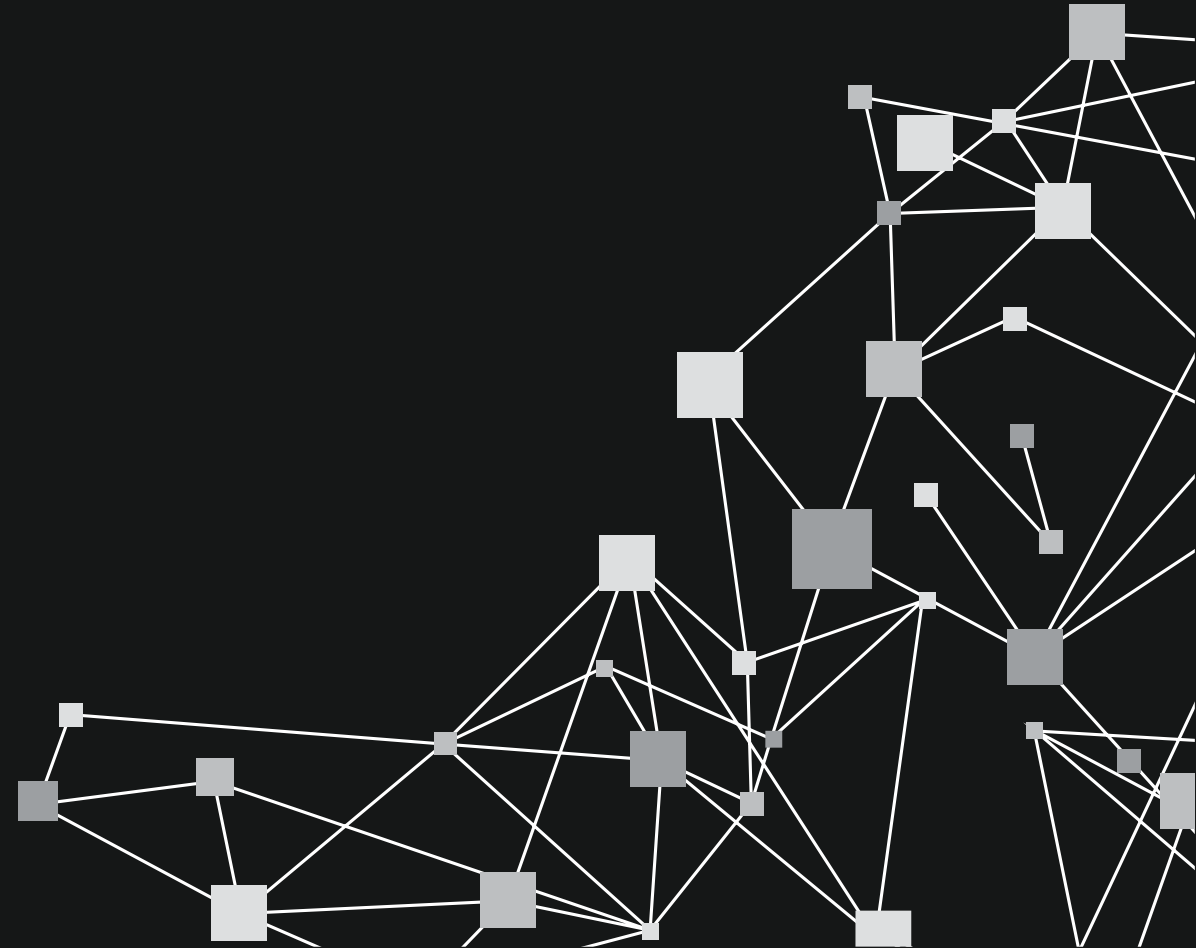
www.hpi.de



Motivation

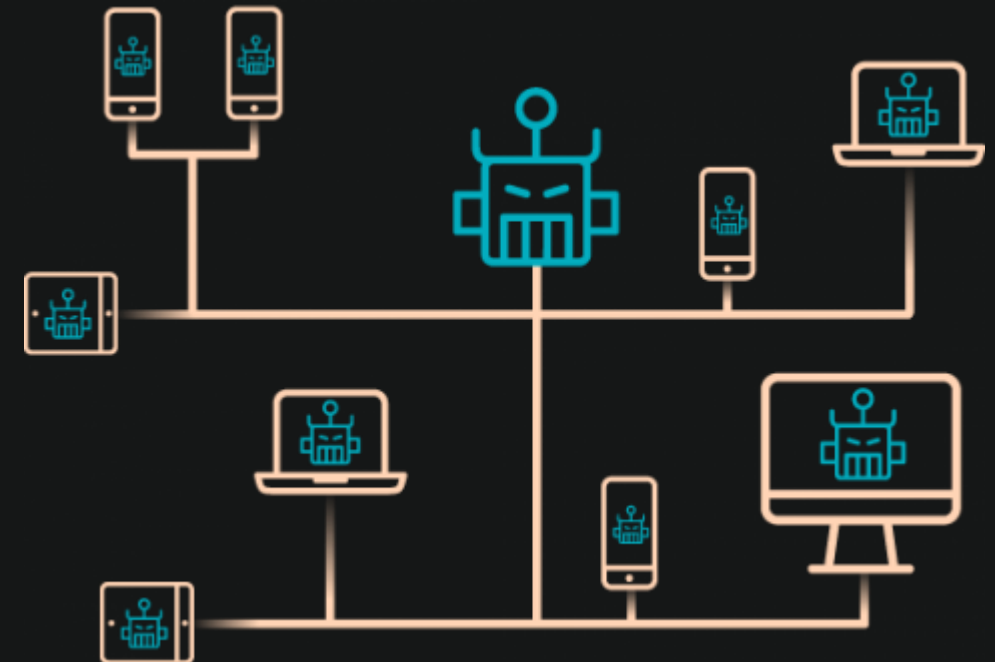
**Design IT.
Create Knowledge.**

www.hpi.de



Mirai Botnet — Massive DDoS Attacks (2016)

- Scanned for IoT devices with default credentials
 - Added each to botnet
- botnet grew to hundreds of thousands of devices
 - record-setting DDoS attacks
 - outages for Twitter, Netflix, Reddit...



<https://www.myrasecurity.com/de/knowledge-hub/mirai/>

McDonald's AI Chatbot Default Password Exposure (2025)



- third-party hiring chatbot used by McDonald's retained a default admin password
 - access the system and view personal data of millions of applicants
 - exposing 64 million+ profiles

“the McHire administration interface for restaurant owners accepted the default credentials 123456:123456”

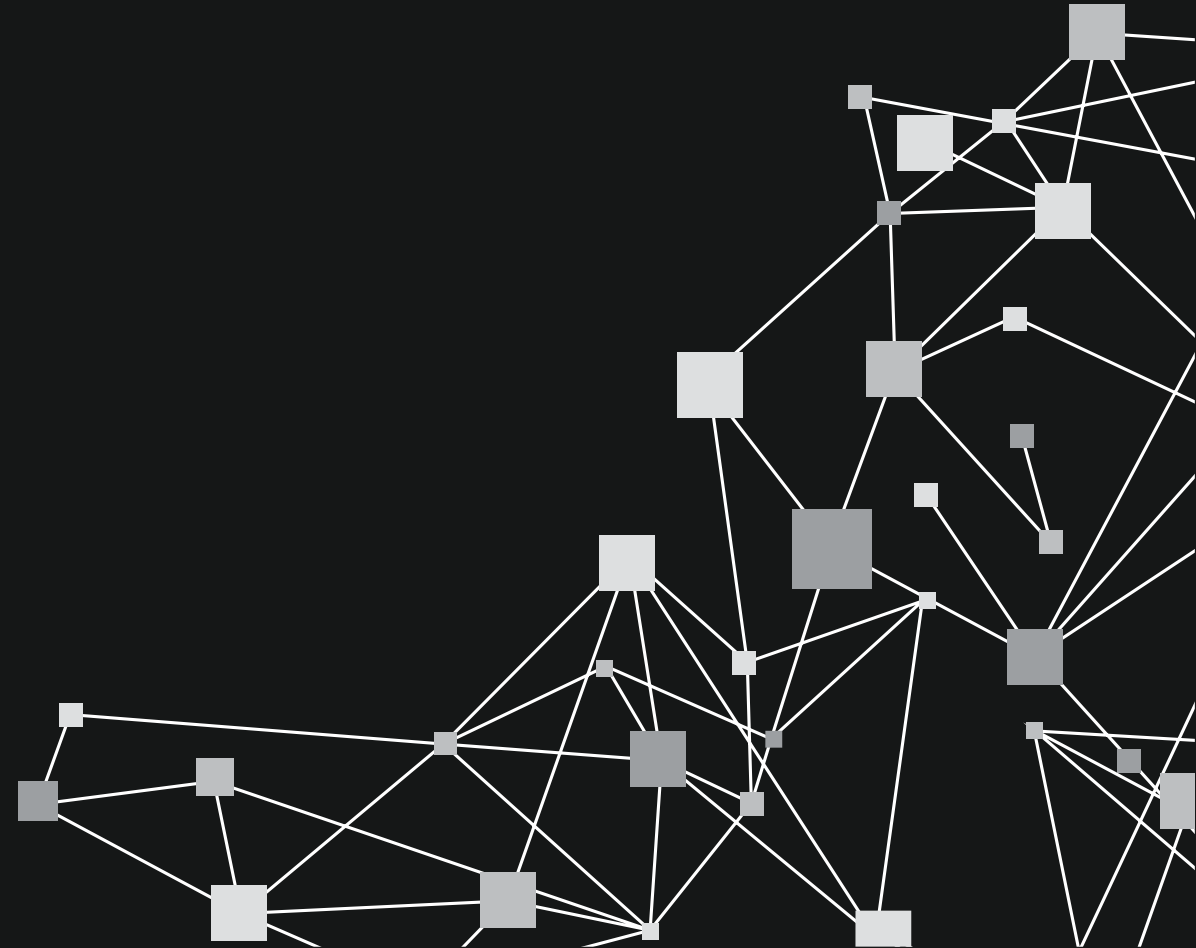


<https://www.news.com.au/technology/online/hacking/dystopian-mcdonalds-ai-chat-bot-olivia-hacked-with-simple-password/news-story/2c0f09f9fb0396f6ca17c56419133fee>

State of the Art

**Design IT.
Create Knowledge.**

www.hpi.de



Default Credential Lists



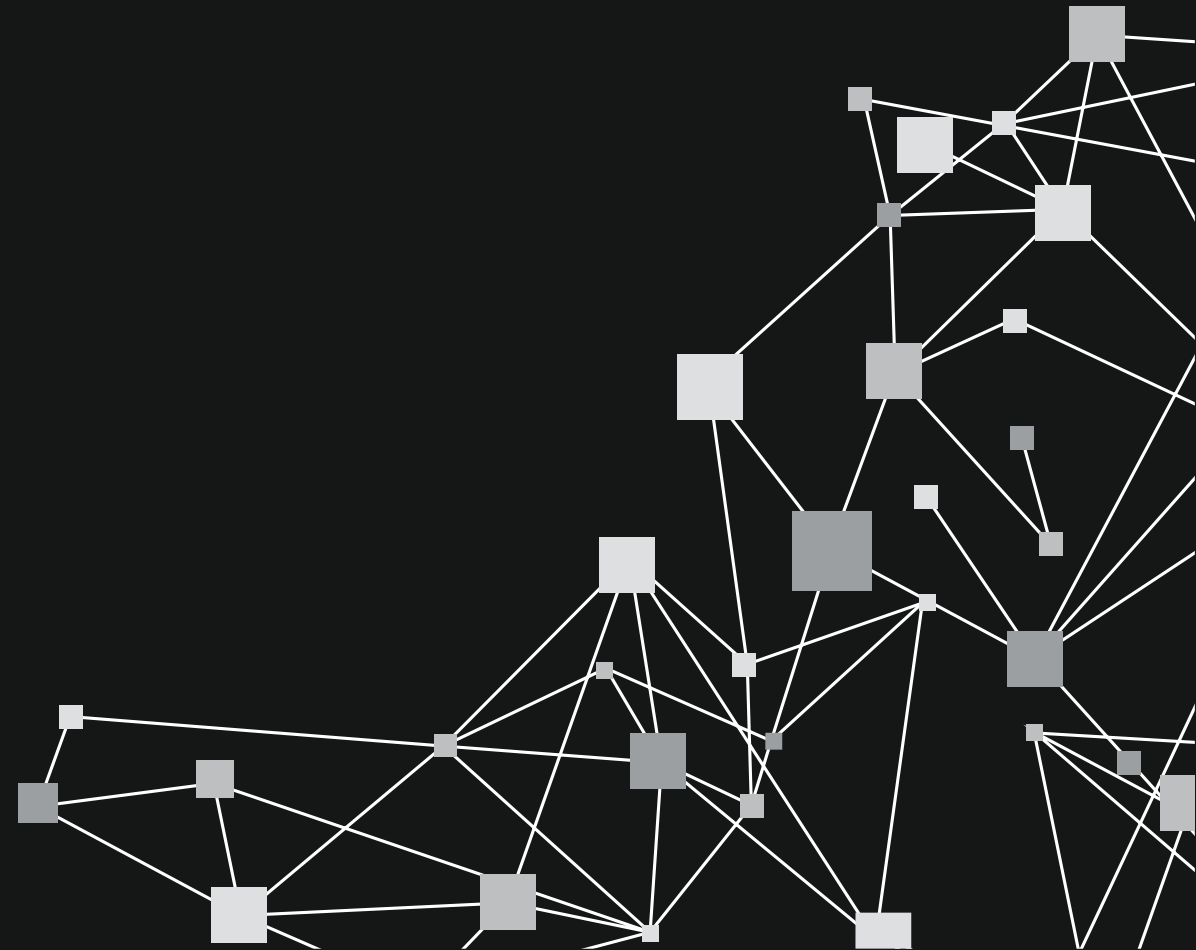
- <https://default-password.info/>
- <https://github.com/danielmiessler/SecLists>
- <https://www.routerpasswords.com/>
- ...

- Nmap http-default-accounts script
- Metasploit
- Vulnerability scanners (Nessus, OpenVAS, etc.)
- All rely on manually maintained lists

Project

**Design IT.
Create Knowledge.**

www.hpi.de

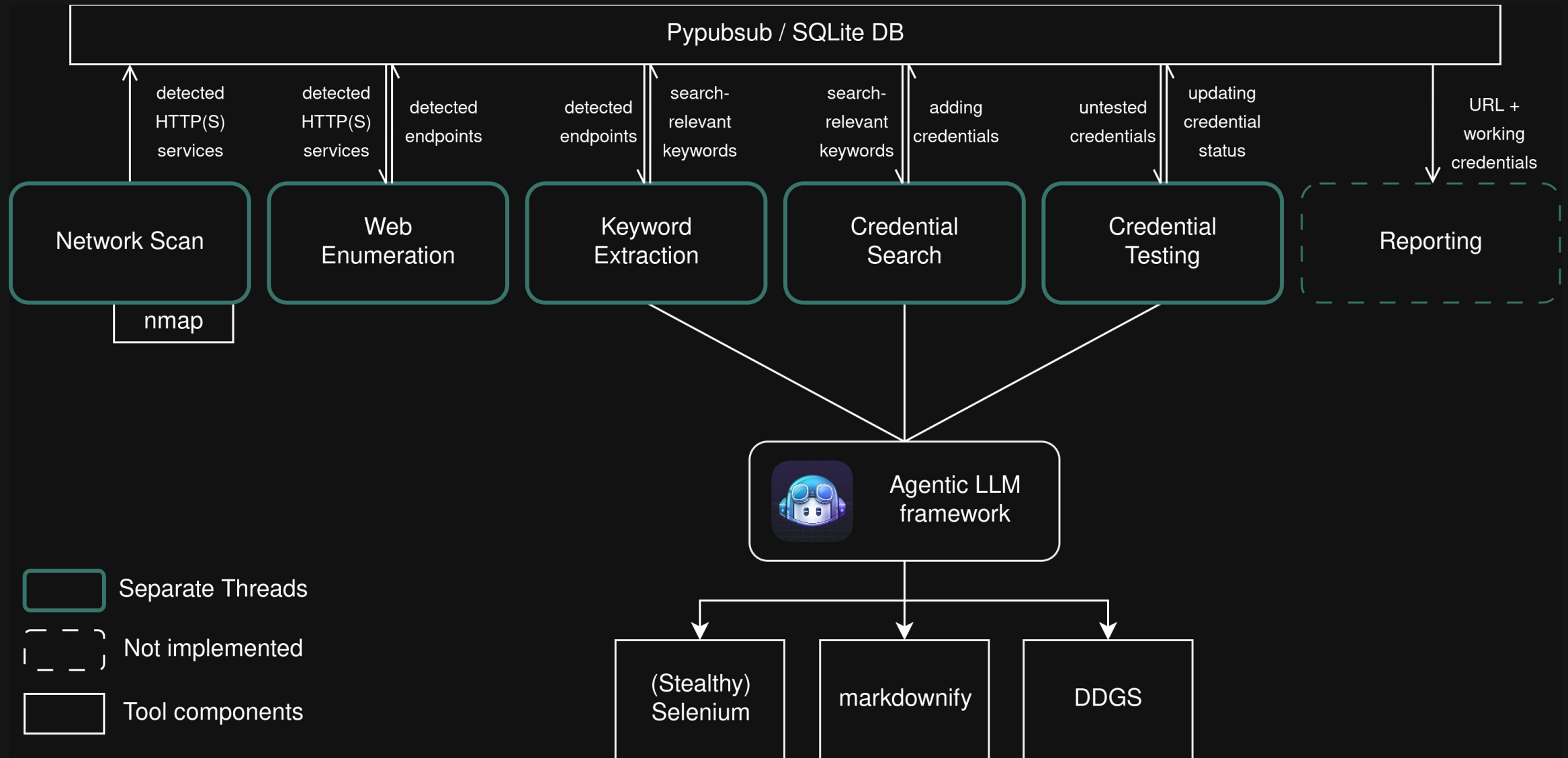


Main Idea



- Cover diverse applications using LLMs
 - Find credentials using agentic online search
 - Test credentials using agentic browser automation
- Build long-running scanner
 - Stateful
 - Runs continuously in background

Architecture



- Keep persistence and cache data
 - Want to persist over restarts/crashes
 - LLM calls are expensive (time & money)
 - Avoiding unnecessary network traffic
- Solution: SQLite database
 - Python package “peewee” allows easy database integration
- Structures communication between models



```
class Service(BaseModel):
    """A discovered service with host, port and https true/false"""
    pk = pw.AutoField()
    host = pw.CharField(max_length=64)
    port = pw.IntegerField()
    https = pw.BooleanField()
    enum_in_progress = pw.BooleanField(default=False)
    _credentials = pw.TextField(null=True, default=None) # None means not searched yet
```

```
class Endpoint(BaseModel):
    """A path on a service that responded with a non-error status code"""
    pk = pw.AutoField()
    service = pw.ForeignKeyField(Service, backref='endpoints')
    path = pw.CharField()
    is_login = pw.BooleanField(default=False)
    page_source = pw.TextField() # raw HTML of the page at this endpoint
    # keywords from the page at this endpoint; None means not analyzed yet
    _keywords = pw.TextField(null=True, default=None)
    _tested_credentials = pw.TextField(default="[]") # for login panels only
    working_credentials = pw.CharField(default="")
```

Architecture – Future Modules

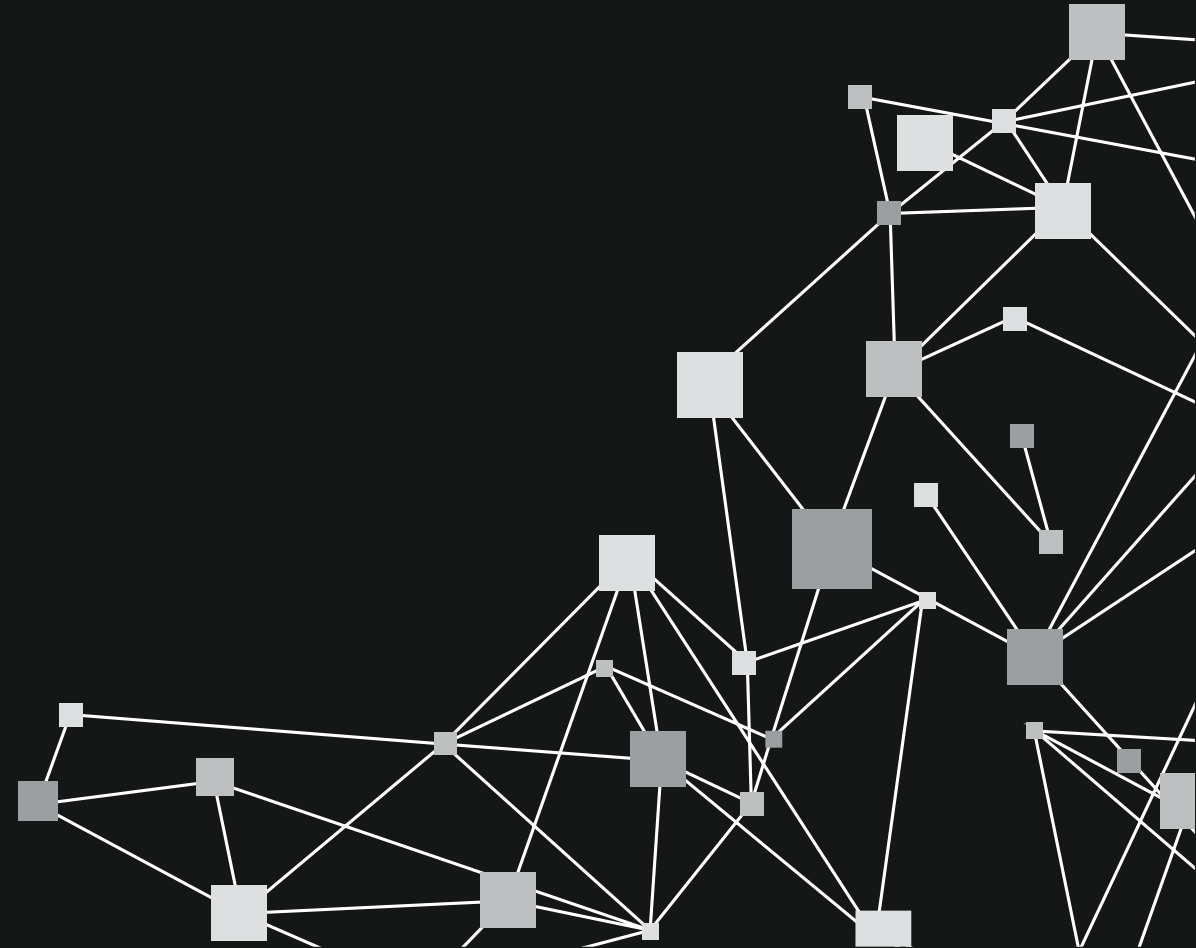


- Reporting module
 - Informs admin about successful logins
- Subdomain enumeration
 - Some webapps only accessible with correct (sub)domain
 - There are tools for that
 - Not implemented yet

Difficulties (so far)

**Design IT.
Create Knowledge.**

www.hpi.de



Difficulties – Finding Login Panels

- Invalid / Unreliable status codes
 - Server might always respond 200
 - Solution: query known invalid and compare with fuzzy-matching
- Client-side rendering
 - Single Page Applications might build DOM dynamically
 - Solution (TODO): Use Selenium instead of simple http requests

Difficulties – Finding Login Panels

- Returning same page for many routes
 - Server might always respond with login panel if unauthenticated
 - Solution (TODO): fuzzy deduplication
- Subdomains
 - Multiple servers can share one port via reverse proxy
 - Solution (TODO): Fuzz "Host" header

Difficulties - AI



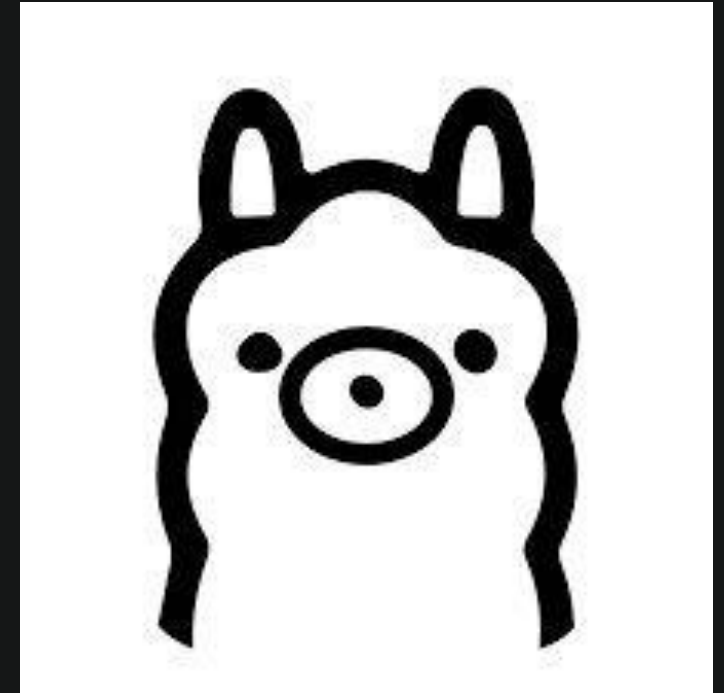
- With remote API
 - rate limiting
 - Parallel accesses can trigger "secondary rate limiting"
 - Solution: serialize API calls and respect rate limiting headers
 - sharing sensitive information
 - can be expensive



Difficulties - AI



- With self-hosting (e.g., Ollama)
 - Very slow
 - GPU/NPU might not be available
 - Limited (V)RAM
 - Should assume conservative amount



Difficulties - AI

- Exceeding context window (CW) or max input tokens
 - Small LLMs limited to 32k – 128k CW
 - Github API limits to only 8k!
- => Context engineering required



Context Engineering

- Websites often large
 - o Convert to markdown
 - o Remove unnecessary information (e.g., scripts)
 - o Depending on goal: summarization in chunks



Difficulties – LLM gets lost / forgets the task

- Good prompts are key
 - Bad: “Find default credentials for mailcow”
 - Can result in “I don’t know” or “I am not allowed to tell you that”
 - The LLM gives up quickly or isn’t using the provided tools

Difficulties – LLM gets lost / forgets the task



- Our current credential search prompt:

```
PROMPT_TEMPLATE = """
```

```
You are a blueteam pentester trying to find default credentials for an unknown web application.  
From the web applications pages you have gathered the following keywords/links for a web search:  
%s
```

```
Follow these steps to find the credentials:
```

- ```
1. Identify the official website or repository for the application.
2. Crawl the site or repository for any documentation, installation guides, or configuration files that might contain
 default credentials. Follow links if necessary.
3. Submit the found credentials using the submit_credentials tool.
```

```
Search the web using carefully chosen, descriptive queries.
```

```
Use the fetch_url tool to retrieve the content of promising links.
```

```
If you find unrelated content, try refining your search.
```

```
You may use tools multiple times. Do not give up quickly.
```

```
Submit the credentials using the submit_credentials tool once you find them.
```

```
"""
```

# Difficulties – Detecting Successful Login

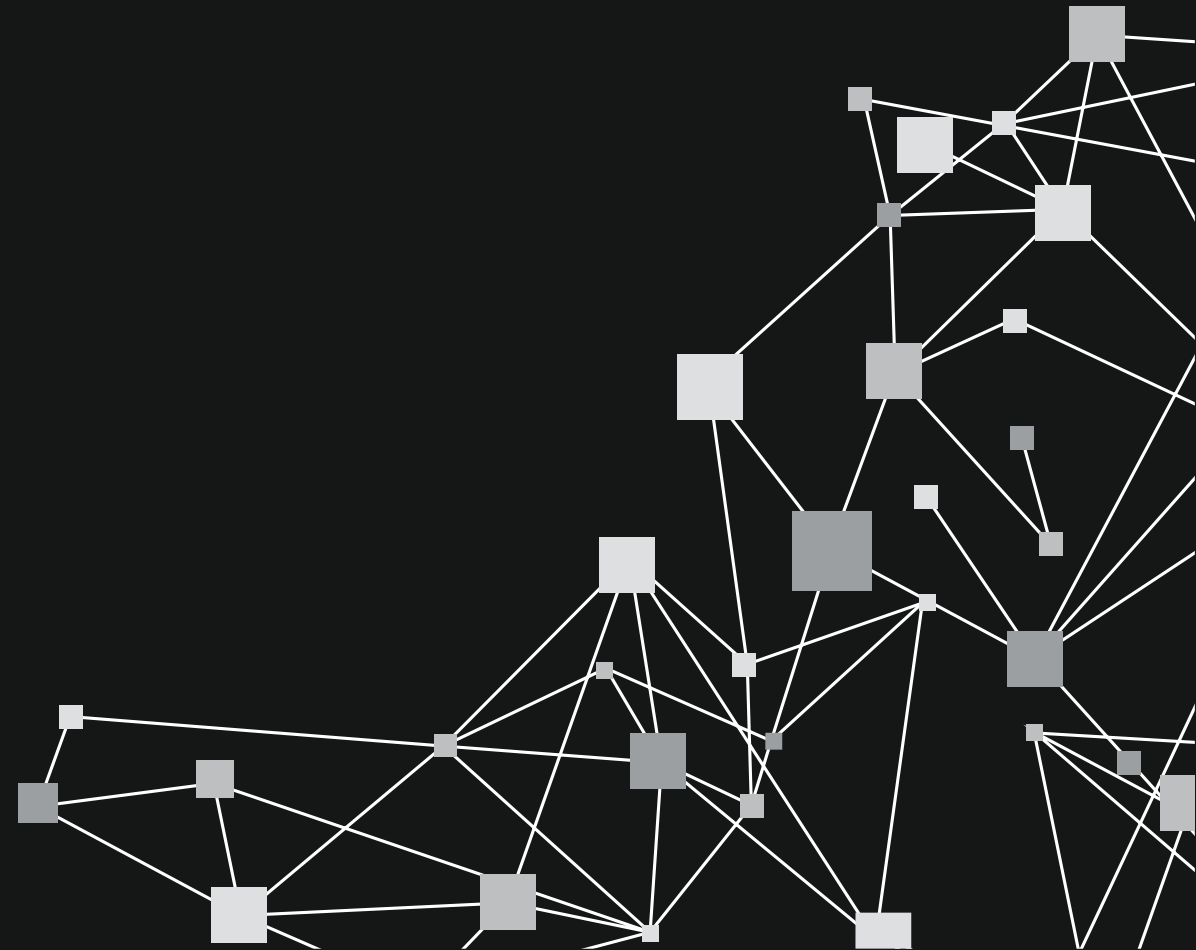
- Heuristic to determine if login was successful
  - Redirection changes path
  - Content changes significantly
- False Positives!
  - Solution: record baseline using “known” wrong password + Cookies

```
login_successful = (
 (before_path != after_path) or
 (simhash.Simhash(before_page_source).distance(simhash.Simhash(after_page_source)) > 10)
)
```

# Live Demo

**Design IT.  
Create Knowledge.**

[www.hpi.de](http://www.hpi.de)



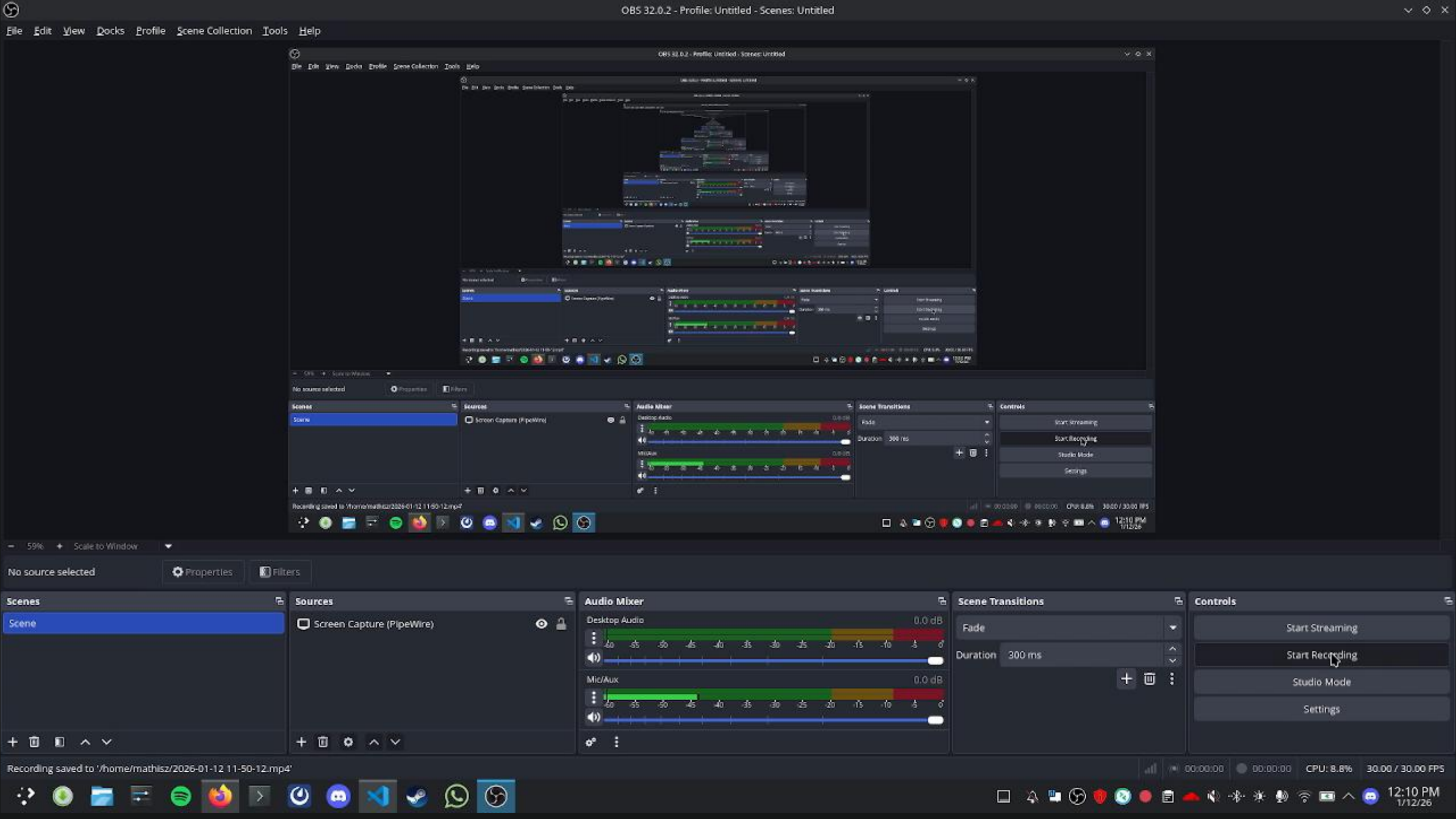


# MailCow



- Default credentials: admin:mohoo
- <https://localhost:80>





| Week          | Tasks                                                             |
|---------------|-------------------------------------------------------------------|
| 19.01.-25.01. | Fully automated testing via CI/CD<br>+ 20 apps with default creds |
| 26.01.-01.02. | Try and evaluate other methods/prompts using the testing<br>setup |
| 02.02.-08.02. | Implement persistence/long time scanning                          |
| 09.02.-15.02. | Evaluation + Buffer                                               |
| 16.02.-22.02. | Final Presentation                                                |

TESTING FUNCTIONAL DEADLINE

# Q&A

Do you agree with our concept, or would you approach it differently?

**Design IT.  
Create Knowledge.**

[www.hpi.de](http://www.hpi.de)

